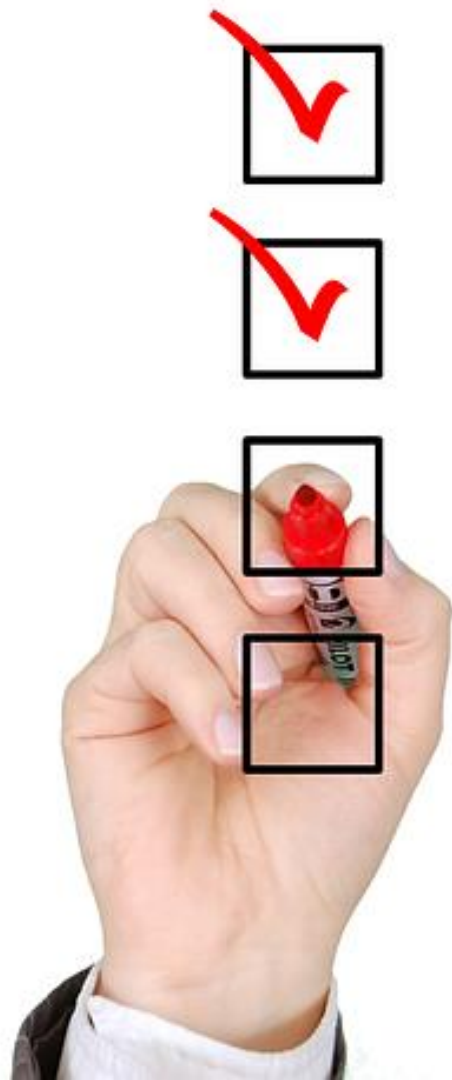


BANCO DE DADOS NÃO RELACIONAL

Agregações e Otimização de Consultas no MongoDB

Tópicos da Aula



- ❑ **Objetivo:**

- Aprender a utilizar o **Aggregation Framework**
- Implementar pipelines de agregação para análise de dados
- Aplicar técnicas de otimização de consultas para melhorar a performance

- ❑ **Exercícios Práticos**

Inserir documentos na coleção alunos

```
db.alunos.drop()
db.alunos.insertMany([
  { nome: "Pedro", idade: 25, genero: "M" },
  { nome: "Ana", idade: 20, genero: "F" },
  { nome: "Maria", idade: 21, genero: "F" },
  { nome: "Lucas", idade: 28, genero: "M" },
  { nome: "João", idade: 22, genero: "M" },
  { nome: "Renata", idade: 24, genero: "F" },
  { nome: "Paulo", idade: 23, genero: "M" },
  { nome: "Bruna", idade: 27, genero: "F" },
  { nome: "Irene", idade: 20, genero: "F" },
  { nome: "Roberto", idade: 21, genero: "M" },
  { nome: "Yuri", genero: "M" }
])
```

Introdução às Agregações

- ❑ No MongoDB, agregação é um processo de processamento de dados semelhante às funções SQL como GROUP BY, HAVING e SUM.
- ❑ O método aggregate() é usado para executar operações de agregação em documentos de uma coleção.
- ❑ O aggregate() aceita um array de estágios de um pipeline que processam e transformam os documentos conforme especificado, produzindo um conjunto de resultados.
O *aggregation pipeline* permite processar dados de forma eficiente e estruturada.
- ❑ Cada etapa do pipeline aplica uma operação nos documentos, transformando-os progressivamente.

Introdução ao Aggregation Framework

Conceitos:

- O **Aggregation Framework** permite realizar transformações complexas nos dados armazenados
- Funciona em **estágios**, onde cada etapa processa e modifica os documentos
- Permite filtrar, agrupar, ordenar e calcular valores agregados

❑ Exemplo:

```
// Exibir a quantidade de estações por localidade
db.estacoes_meteorologicas.aggregate([
  { $group: { _id: "$localizacao", total: { $sum: 1 } } }
]);
```

Explicação:

1. **\$group** agrupa os documentos com base no campo **localizacao**
2. **\$sum** conta quantas estações existem em cada localidade

O que é o Aggregation Pipeline?

- ❑ O *Aggregation Pipeline* é composto por múltiplos estágios, cada um aplicando uma transformação aos dados.
- ❑ Esses estágios são encadeados para processar os documentos de uma coleção.
- ❑ Alguns operadores comuns:
 - ❑ **\$match**: Filtra documentos (como um WHERE no SQL).
 - ❑ **\$group**: Agrupa documentos com base em um campo e aplica funções como count(), sum(), avg().
 - ❑ **\$sort**: Ordena os resultados.
 - ❑ **\$limit**: Restringe o número de documentos retornados.
 - ❑ **\$project**: Define quais campos devem ser retornados e pode criar novos campos calculados.
 - ❑ **\$skip**: pula um número específico de documentos. Semelhante ao termo offset nos comandos SQL;

O que é o Aggregation Pipeline?

- ❑ O resultado é um conjunto de documentos transformados conforme os estágios do pipeline.
- ❑ O método **aggregate()** é especialmente útil para realizar cálculos de agregação complexos, como calcular **médias (\$avg)**, **mínimos (\$min)**, **máximos (\$max)**, **somas/contar (\$sum)** e outros tipos de análises de dados.
- ❑ Ele oferece uma flexibilidade significativa para manipular e transformar dados diretamente no BD.

Aggregation Framework e Aggregation Pipeline

- ❑ **Aggregation Framework** e **Aggregation Pipeline** estão relacionados, mas não são exatamente a mesma coisa.
- ❑ Vejamos:
 - **Aggregation Framework** é o conceito maior que engloba diferentes formas de agregação.
 - **Aggregation Pipeline** é uma das principais ferramentas dentro do Aggregation Framework e funciona como um conjunto de estágios encadeados para transformar dados.

Conceito:

O **\$match** filtra documentos conforme uma condição, funcionando como um WHERE no SQL.

Exemplo:

```
{ "$match": { "categoria": "Eletrônicos" } }
```

Explicação:

- Esse estágio seleciona apenas os documentos onde a chave categoria é igual a **Eletrônicos**.
- O uso de **\$match** logo no início do pipeline melhora a performance, reduzindo o volume de dados processados nos estágios seguintes.

Conceito:

O **\$group** agrupa documentos pelo valor de um campo e permite aplicar funções como soma, média e contagem.

Exemplo:

```
{ "$group": { "_id": "$categoria", "totalProdutos": { "$sum": 1 } } }
```

Explicação:

- Agrupa os documentos pelo campo categoria.
- Conta quantos documentos existem em cada categoria (\$sum: 1).

Exemplo de \$match e \$group

- o método `aggregate` recebe um array com os estágios `match` e `group`, nesta ordem, pois a ordem dos estágios influencia na resposta. Neste caso, é necessário seguir a ordem dos estágios no pipeline de agregação.
- O comando soma a *idade* de todos os documentos.

```
db.alunos.aggregate([
  {
    $match: {
      genero: "M"
    }
  },
  {
    $group: {
      _id: null,
      soma: { $sum: "$idade" }
    }
  }
])
```

- \$match:** filtra os documentos que possuem o campo *genero* igual a M;
- \$group:** agrupa os documentos seguindo as operações de agregação:
- _id:** *null* é obrigatório. Indica o campo que queremos agrupar. *null* significa que estamos agrupando
- todos os documentos em um único conjunto;
- "\$idade"** é o campo que estamos usando para calcular o somatório.

Exemplo de \$match e \$group

- No exemplo a seguir, o estágio **\$match** é aplicado em dois momentos:

```
db.alunos.aggregate([
  {
    $match: {
      idade: { $lt: 25 }
    } },
  {
    $group: {
      _id: "$genero",
      quantidade: { $sum: 1 }
    } },
  {
    $match: {
      quantidade: { $eq: 4 }
    } },
  {
    $project: {
      sexo: "$_id",
      quantidade: 1,
      _id: 0
    }
  }
])
```

1)

- **Antes do estágio \$group:**
 - ele faz o papel de **where** (no comando SQL), ou seja, o filtro não pode usar campos calculados no estágio *\$group*;
- **Após o estágio \$group:**
 - ele faz o papel de **having** (no comando SQL), ou seja, o filtro só pode usar campos calculados no estágio *\$group*.
- No exemplo a seguir o filtro **quantidade = 4** faz uso do campo *quantidade* que é calculado no estágio *\$group*.

Exemplo de \$match e \$group

- Observe que o estágio **\$match** faz o papel de **where** (\$match antes do agrupamento) e **having** (\$match após o agrupamento) no comando SQL:

```
db.alunos.aggregate([
  {
    $match: {
      idade: { $lt: 25 }
    } },
  {
    $group: {
      _id: "$genero",
      quantidade: { $sum: 1 }
    } },
  {
    $match: {
      quantidade: { $eq: 4 }
    } },
  {
    $project: {
      sexo: "$_id",
      quantidade: 1,
      _id: 0
    }
  }
])
```

```
select sexo, quantidade
from alunos
where idade < 25
group by genero
having quantidade = 4;
```

Operador

Conceito:

O **\$sort** ordena os documentos, equivalente ao **ORDER BY** no SQL.

Exemplo:

```
{ "$sort": { "preco": -1 } }
```

Explicação:

Ordena os documentos pelo campo **preco** em ordem decrescente (-1).
Para ordem crescente, usamos 1.

- ❑ **Exemplo de \$group e \$sort:** o método `aggregate` recebe um array com os estágios `group` e `sort`, nesta ordem, para agrupar os documentos pelo campo `genero` e depois calcular a média no campo `idade`.

```
db.alunos.aggregate([
  {
    $group: {
      _id: "$genero",
      media: { $avg: "$idade" }
    },
    {
      $sort: { media: -1 }
    }
  ])
```

O resultado terá dois documentos, visto que a coleção possui documentos com os gêneros F e M.

Significado dos termos:

- **\$match:** filtra os documentos que possuem o campo `genero` igual a M;
- **\$group:** agrupa os documentos seguindo as operações de agregação:
- **_id:** `"$genero"` significa que estamos agrupando todos os documentos pelo valor do campo `genero`;
- `"$idade"` é o campo que estamos usando para calcular a média.

Operador

Conceito:

O **\$project** define quais campos serão retornados no resultado final.

Exemplo:

```
{ "$project": { "nome": 1, "preco": 1, "_id": 0 } }
```

Explicação:

Retorna apenas os campos **nome** e **preco**, omitindo o campo **_id**.

- ❑ **Exemplo de \$project:** o exemplo a seguir usamos o estágio **\$project** para renomear o **campo _id** para **sexo** no documento de saída. O estágio **\$project** é adicionado após o estágio **\$sort**.

```
db.alunos.aggregate([
  {
    $group: {
      _id: "$genero",
      soma: { $sum: "$idade" }
    },
    {
      $sort: {
        soma: -1
      },
      {
        $project: {
          sexo: "$_id", soma: 1, _id: 0
        }
      }
    }
  ])
```

Significado dos termos:

- **sexo:** "\$_id": renomeamos o campo **_id** para **sexo**;
- **soma:** 1: o valor 1 ou true indica que mantemos o campo **soma** no documento de saída;
- **_id:** 0: o valor 0 ou false indica que excluimos o campo **_id** no documento de saída.

Conceito:

O **\$limit** restringe o número de documentos retornados, equivalente ao **LIMIT** no SQL.

Exemplo:

```
{ "$limit": 5 }
```

Explicação:

Retorna apenas os **primeiros 5 documentos** do pipeline.

Exemplo

```
[  
  { "$match": { "categoria": "Eletrônicos" } },  
  { "$group": { "_id": "$marca", "total": { "$sum": 1 } } },  
  { "$sort": { "total": -1 } }, { "$limit": 3 }  
]
```

Explicação:

1. Filtra os documentos da categoria **Eletrônicos**.
2. Agrupa os produtos pela **marca** e conta quantos existem.
3. Ordena os resultados de forma **decrescente**.
4. Retorna apenas os **3 primeiros resultados**.



BANCO DE DADOS NÃO RELACIONAL

Índices e Otimização de Consultas no MongoDB

Índices e Otimização de Consultas

Conceitos:

- Índices melhoram o desempenho de buscas ao permitir acesso rápido aos documentos
- Um índice pode ser criado em um ou mais campos para acelerar as consultas
- MongoDB usa índices automaticamente se disponíveis

❑ Exemplo:

// Criar um índice no campo "localizacao"

```
db.estacoes_meteorologicas.createIndex({ "localizacao": 1 });
```

Explicação:

- Esse índice melhora a velocidade de buscas que filtram por **localizacao**



BANCO DE DADOS NÃO RELACIONAL

Atividade Prática (Individual)

Atividade Prática (Entrega de Requisitos ABP)

Exercícios Práticos Individuais

1. Filtrar todas as estações localizadas em "São Paulo".
2. Contar quantas estações possuem o sensor "temperatura".
3. Listar as cinco estações mais recentes cadastradas.
4. Criar um índice no campo "sensores".
5. Contar quantas estações existem por cada tipo de sensor.

Atividade Prática em Grupo (ABP)

Tarefa:

- Criar uma consulta que exiba a média de temperatura de todas as estações por localidade
- Implementar um índice que otimize a busca de alertas meteorológicos
- Criar um relatório que exiba a quantidade de alertas emitidos por mês
- Apresentar os resultados ao professor

Requisitos do Projeto:

- Implementação de agregações para cálculos de médias (**MBD.03**)
- Otimização de consultas com índices (**MBD.04**)
- Implementação de contagens e agrupamentos eficientes (**MBD.05**)

Referências Bibliográficas

❑ Material de apoio:

- Chodorow, Kristina. *MongoDB: The Definitive Guide*. O'Reilly Media, 2013.
- *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs/>
- *MongoDB Documentation*. Disponível em: <https://www.mongodb.com/docs/>
- Cattell, Rick. *Scalable SQL and NoSQL Data Stores*. ACM, 2011.
- *Mais detalhes sobre operadores:*
<https://www.mongodb.com/docs/manual/reference/operator/query>
- *Operadores de atualização:*
(<https://www.mongodb.com/docs/manual/reference/operator/update/#update-operators-1>)
- [Documentação MongoDB CRUD](#)
- [MongoDB Aggregation Framework](#)
- [MongoDB Indexação e Performance](#)

Bibliografia Básica

- ❑ BOAGLIO, Fernando. **MongoDB**: Construa novas aplicações com novas tecnologias. São Paulo: Casa do Código, 2015.
- ❑ ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 7ed. São Paulo: Pearson, 2019.
- ❑ SADALAGE, P.; FOWLER, M. **Nosql Essencial**: Um Guia Conciso Para o Mundo Emergente da Persistência Poliglota. São Paulo: Novatec, 2013.
- ❑ SINGH, Harry. **Data Warehouse**: conceitos, tecnologias, implementação e gerenciamento. São Paulo: Makron Books, 2001.

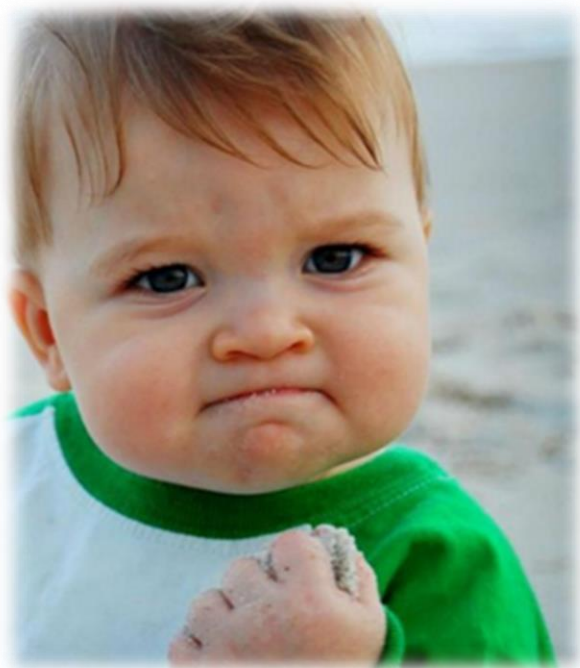
Bibliografia Complementar

- ❑ FAROULT, Stephane. **Refatorando Aplicativos SQL**. Rio de Janeiro: Alta Books, 2009.
- ❑ PANIZ, D. **NoSQL**: Como armazenar os dados de uma aplicação moderna. Casa do Código, 2016.
- ❑ SOUZA, M. **Desvendando o MongoDB**. Rio de Janeiro: Ciência Moderna, 2015.

Dúvidas?



Considerações Finais



**Professor(a):
Lucineide Pimenta**

Bom descanso à todos!

